DAA Assignment 4 — Dynamic Programming and String Search

Alua Rakhimzhanova · SE-2509

Experiment Design Card

I tested how input length and input type affect Edit Distance and string search. For Edit Distance, I compared recursion, full-table DP, and two-row DP on strings of lengths 2, 4, 6, 8, and 10. I also compared the two DP versions at lengths from 128 to 2,048. The input pairs were identical, disjoint, or random. For string search, I compared Naive Search and KMP on random text, near misses, and inputs with overlapping matches. I varied text length with a fixed pattern length of 32, then varied pattern length with a fixed text length of 160,000.

Each case had three warmup runs and five measured runs. The program generated random inputs with seed 2509 and gave the same input to both algorithms in each comparison. I used the median of five runs for runtime. I also counted recursive calls, computed DP cells, allocated integer slots, and character comparisons. The KMP comparison count includes both LPS construction and search.

I expected recursion to grow quickly on disjoint strings, two-row DP to save storage, and KMP to make fewer comparisons on near misses. I expected a smaller difference on random text because Naive Search often stops at an early mismatch.

Edit Distance Results

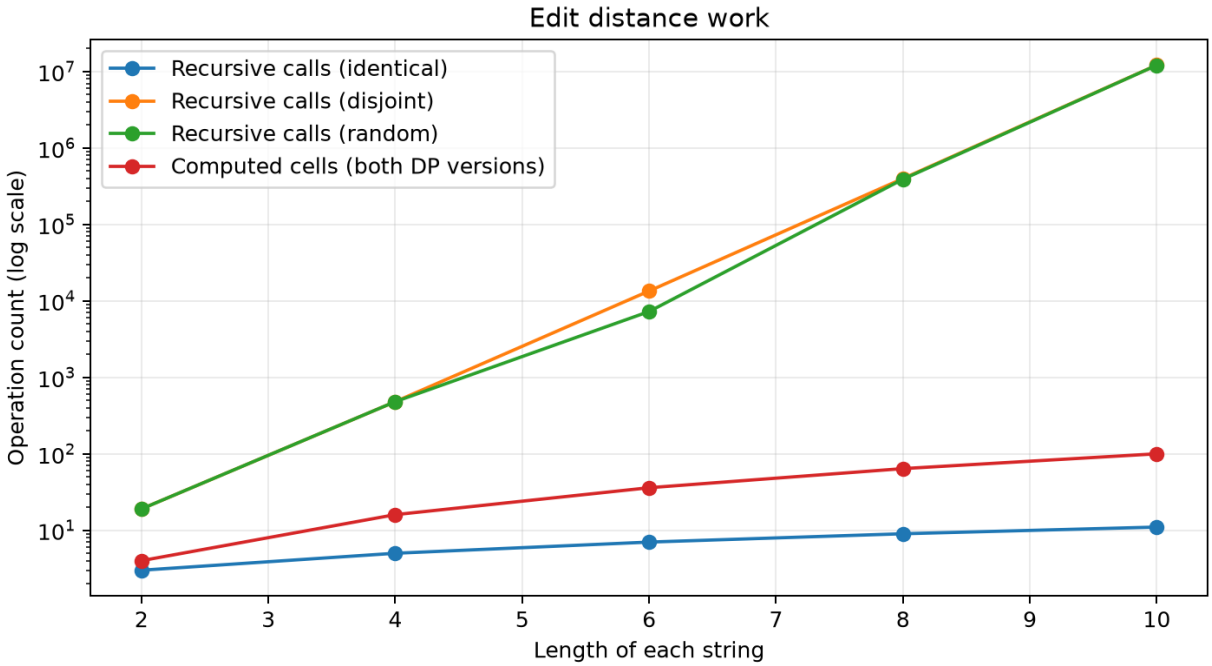


Figure 1. Recursive calls grow quickly on disjoint strings, while identical strings need only one recursive call per step. Both DP versions compute the same number of interior cells.

The recursion probe completed at length 11 in **450.974 ms** with **67,570,078 calls**. At length 12, the Java process exceeded the two-second limit. This result supports the prediction that plain recursion becomes expensive even on short disjoint strings. The timeout applies to the whole Java process, while the recorded time measures the algorithm call.

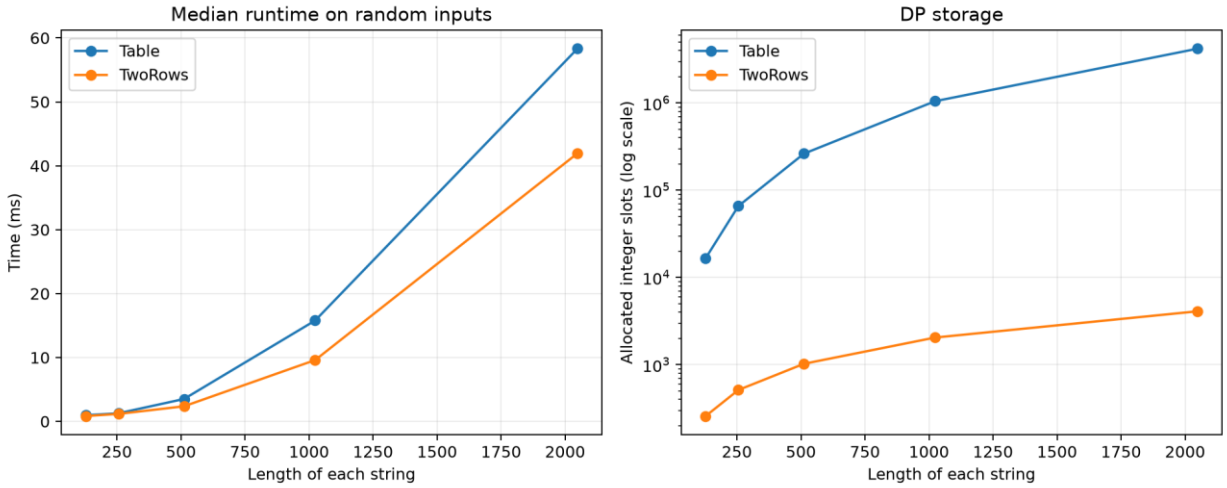


Figure 2. Two-row DP reduces allocated storage while computing the same number of interior cells.

For random strings of length 2,048, both DP versions computed **4,194,304 cells**. The full table allocated **4,198,401 integer slots** and took **58.314 ms**. Two-row DP allocated **4,098 slots** and took **41.938 ms**. These are median times. The two-row version used about **1,025 times fewer integer slots** in this case. It keeps only the previous and current rows, so it can return the distance but does not keep the full table needed for straightforward reconstruction of the edit sequence.

String Search Results

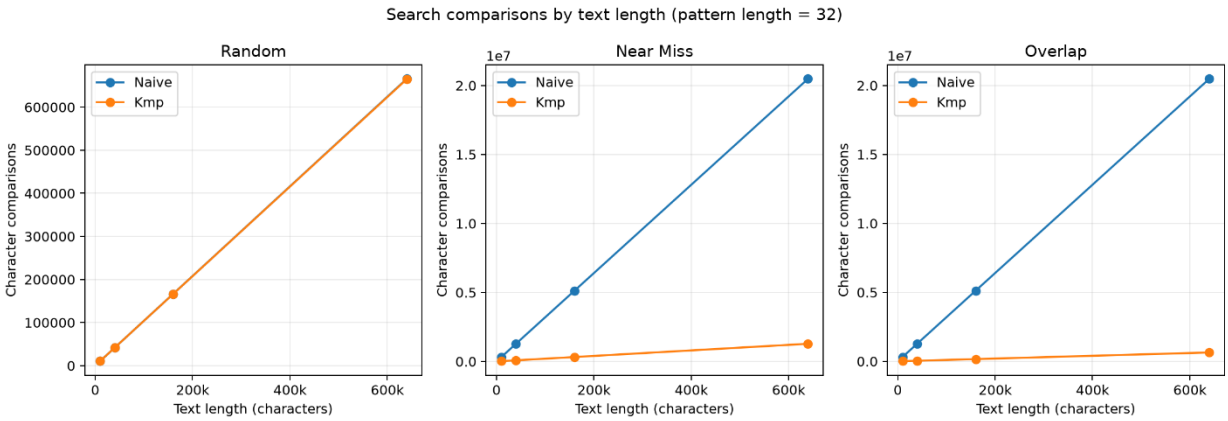


Figure 3. Naive Search and KMP made nearly the same number of comparisons on random text. KMP used fewer comparisons on near misses and overlapping inputs.

At text length 640,000 and pattern length 32, Naive Search made **665,629** comparisons on random text and KMP made **664,727**. Most starting positions failed early. On the near-miss input, Naive Search made **20,479,008** comparisons and KMP made **1,280,030**. Naive Search checked the long matching prefix again at each start. KMP used LPS to keep part of the previous

match. On the overlap input, both algorithms returned **639,969** matches, including overlapping ones.

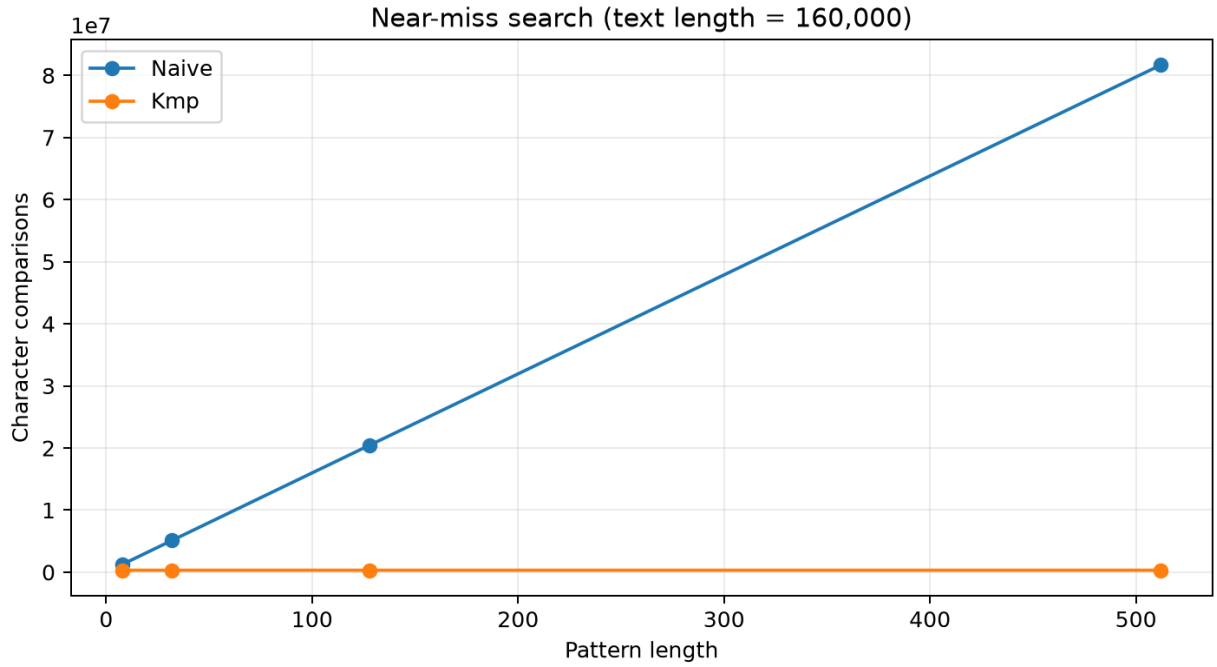


Figure 4. A longer near-miss pattern greatly increases the comparison count for Naive Search. KMP's count stays close to the same level in this experiment.

At text length 160,000 and pattern length 512, Naive Search made **81,658,368** comparisons, compared with **320,510** for KMP. Their median times were **383.015 ms** and **2.707 ms**. For long texts with repeated characters, I would choose KMP because it avoids checking the same long prefix at every starting position. For short or random inputs, Naive Search is a reasonable choice because it is simple and often finds a mismatch early.

Runtime and Conclusion

The timings include the work of the algorithms and their counters. Input generation and CSV writing were outside the timed section. Runtime depends on the JVM and computer, so I used the median to reduce the effect of an unusually slow run. The operation counts show how much work each algorithm did on the same input. Overall, the results supported the predictions: recursion grew quickly on disjoint strings, two-row DP saved storage, and KMP made far fewer comparisons on near misses.